Fundamentos

Discente: Bruno Victor Paiva da Silva

1. **(Exercício 2.1-3 do livro do Cormen) Escreva um pseudocódigo para a busca linear e mostre, usando invariância de laço, que o seu algoritmo está correto**

Pseudocódigo baseado nas características padrão do livro do Cormen

```
LinearSearch(vetor, key)
    for index <- 1 to vetor.length
        if key == vetor[index]
            return index
    return NIL
```

A invariante do laço da busca linear é que a cada iteração do laço for o subvetor $vetor[1 : index - 1]$ não contém a chave key (1).

1. **Inicialização:** antes da iteração $index = 1$, o subvetor vetor[1:0] é vazio, logo a chave key não está contida em $vetor[1 : 0]$, portanto a afirmação (1) está correta.
2. **Manutenção: s**e sabemos que key não está contido no subvetor $vetor[1 : index - 1]$ então durante a iteração verificamos $vetor[index]$:
   a. Se $vetor[index] == key$, então retornamos $index$, logo a afirmação (1) está correta.
   b. Se não for, sabemos que key não está contido em $vetor[1 : index]$, portanto, no inicio da proxima iteração a invariante ainda é válida.
3. **Finalização:** quando o laço terminar ($index == vetor.length + 1$) a invariante vai nos dizer que key não está contido em $vetor[1 : vetor.length]$ e retornar NIL, portanto a afirmação (1) está correta.

Como a invariante foi válida em todas as etapas (inicialização, manutenção e finalização), o algoritmo está correto.

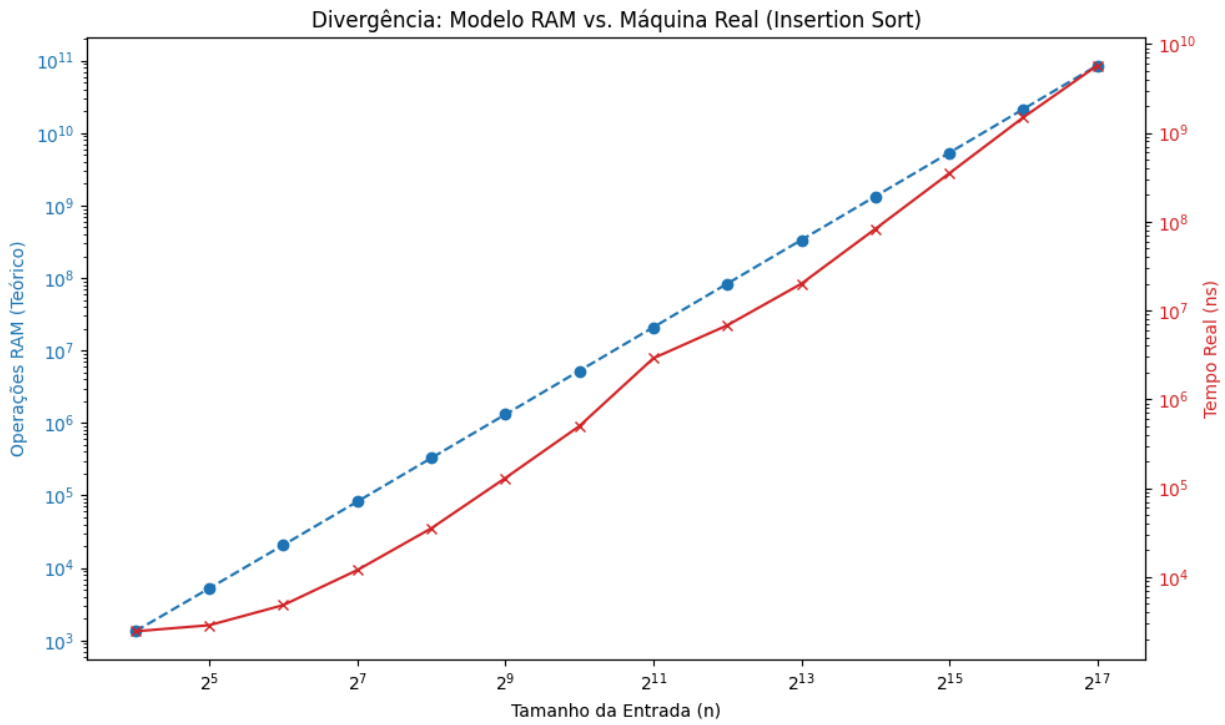
2. Implemente o algoritmo de ordenação por inserção e crie uma cópia anotada dele que mede o número de operações no modelo da Random Access Machine (RAM, seção 2.2 do livro do Cormen). Usando entradas de tamanho crescente, mostre, em um gráfico, quando o tempo de execução no modelo RAM diverge de medições em uma máquina real.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <numeric>
#include <algorithm>

typedef int KeyType;

class RAMTracker {
public:
    unsigned long long total_ops = 0;
    void add(int n) { total_ops += n; }
};

void insertion_sort(std::vector<KeyType>& a) {
    int n = a.size();
    for (int i = 1; i < n; i++) {
        KeyType key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

unsigned long long insertion_sort_instrumented(std::vector<Ke
yType>& a) {
    RAMTracker tracker;
```

```
    int n = a.size();

    tracker.add(1);
    for (int i = 1; i < n; i++) {
        tracker.add(2);

        KeyType key = a[i];
        int j = i - 1;
        tracker.add(3);

        while (j >= 0 && a[j] > key) {
            tracker.add(4);
            a[j + 1] = a[j];
            tracker.add(4);
            j--;
            tracker.add(2);
        }
        tracker.add(1);

        a[j + 1] = key;
        tracker.add(3);
    }
    tracker.add(1);

    return tracker.total_ops;
}

int main() {
    std::cout << "n,ram_ops,real_ns" << std::endl;

    for (int n = 16; n <= 524288; n *= 2) {
        std::vector<KeyType> base_vec(n);
        std::iota(base_vec.begin(), base_vec.end(), 1);
        std::reverse(base_vec.begin(), base_vec.end());

        std::vector<KeyType> v_ram = base_vec;
```

```
        std::vector<KeyType> v_real = base_vec;

        unsigned long long ram_ops = insertion_sort_instrumen
ted(v_ram);

        auto t1 = std::chrono::high_resolution_clock::now();
        insertion_sort(v_real);
        auto t2 = std::chrono::high_resolution_clock::now();

        auto duration = std::chrono::duration_cast<std::chron
o::nanoseconds>
        (t2 - t1).count();

        std::cout << n << "," << ram_ops << "," << duration <
< std::endl;
    }

    return 0;
}
```

Resultados:



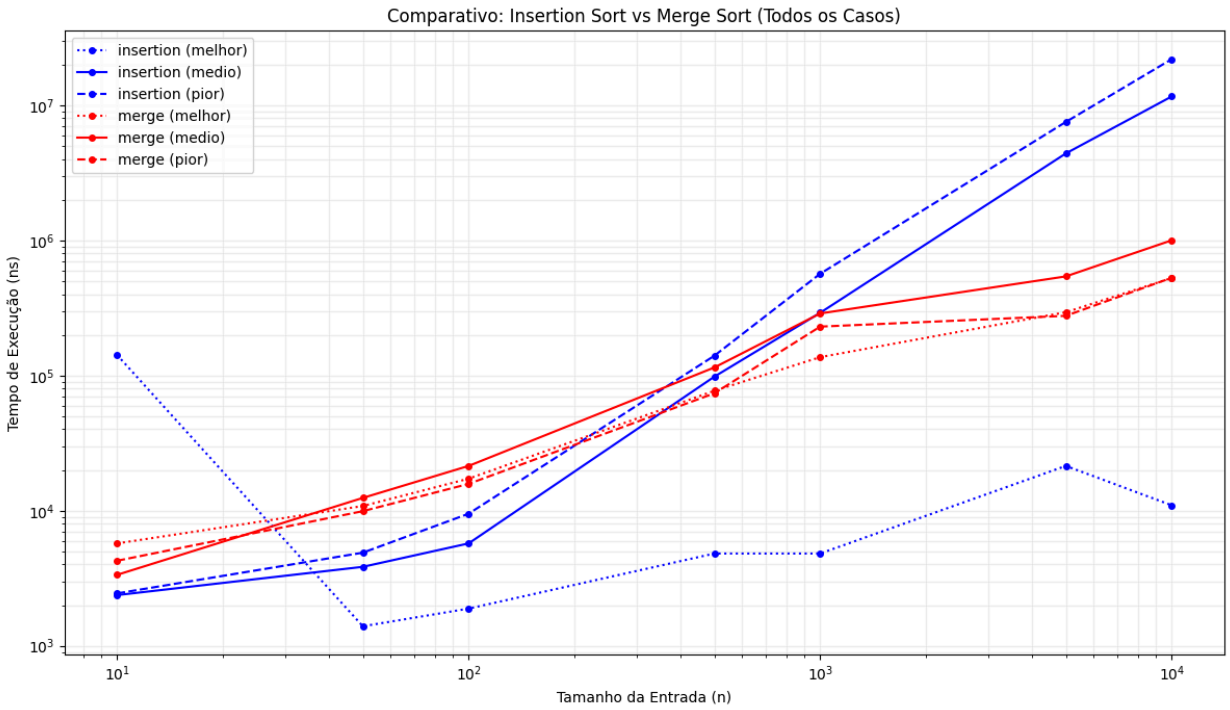
3. Mostre, numericamente, com suas implementações dos algoritmos insertion-sort e merge-sort, como se comporta o desempenho de cada algoritmo em entradas de tamanho crescente, considerando pior, melhor e caso médio. Analise, para cada tipo de entrada, se há algum ponto a partir do qual um algoritmo passa a ser mais rápido do que o outro.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <numeric>
#include <algorithm>
#include <random>

void insertion_sort(std::vector<int>& a) {
    for (int i = 1; i < a.size(); i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
```

```
            j--;
        }
        a[j + 1] = key;
    }
}

void merge(std::vector<int>& a, int p, int q, int r) {
    int n1 = q - p + 1;
    int n2 = r - q;
    std::vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = a[p + i];
    for (int j = 0; j < n2; j++) R[j] = a[q + 1 + j];
    int i = 0, j = 0, k = p;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) a[k++] = L[i++];
        else a[k++] = R[j++];
    }
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}

void merge_sort(std::vector<int>& a, int p, int r) {
    if (p < r) {
        int q = p + (r - p) / 2;
        merge_sort(a, p, q);
        merge_sort(a, q + 1, r);
        merge(a, p, q, r);
    }
}

long long measure(std::vector<int> a, std::string algo) {
    auto t1 = std::chrono::high_resolution_clock::now();
    if (algo == "insertion") insertion_sort(a);
    else merge_sort(a, 0, a.size() - 1);
    auto t2 = std::chrono::high_resolution_clock::now();
    return std::chrono::duration_cast<std::chrono::nanosecond
```

```
s>(t2 - t1).count();
}

int main() {
    std::cout << "n,algo,cenario,tempo_ns" << std::endl;
    std::vector<int> tamanhos = {10, 50, 100, 500, 1000, 500
0, 10000};
    std::mt19937 rng(42);

    for (int n : tamanhos) {
        std::vector<int> melhor(n), pior(n), medio(n);
        std::iota(melhor.begin(), melhor.end(), 1);
        pior = melhor;
        std::reverse(pior.begin(), pior.end());
        medio = melhor;
        std::shuffle(medio.begin(), medio.end(), rng);

        std::string algos[] = {"insertion", "merge"};
        for (auto& algo : algos) {
            std::cout << n << "," << algo << ",melhor,"
                                << measure(melhor, algo) << s
td::endl;
            std::cout << n << "," << algo << ",medio,"
            << measure(medio, algo) << std::endl;
            std::cout << n << "," << algo << ",pior,"
             << measure(pior, algo) << std::endl;
        }
    }
    return 0;
}
```

Resultados:


Comparativo: Insertion Sort vs Merge Sort (Todos os Casos)
insertion (melhor)
insertion (medio)
insertion (pior)
merge (melhor)
merge (medio)
merge (pior)
Tempo de Execução (ns)
Tamanho da Entrada (n)
10^3
10^4
10^5
10^6
10^7
10^1
10^2
10^3
10^4